

Dual-Mode Vision-Guided Robotic Arm — Learning Guide

What to Watch & Study Before Each Phase

How to use this guide:

Before starting any phase of the project, read that phase's section here first.

Watch the listed videos, run the listed practice commands, then move to the actual implementation.

You don't need to learn everything at once — learn per phase, just-in-time.

Table of Contents

1. [Learning Roadmap — Big Picture](#)
2. [Foundations — What You Must Know Before Anything](#)
3. [Phase 0 — ROS2 Fundamentals](#)
4. [Phase 0 — URDF & Robot Description](#)
5. [Phase 0 — Gazebo Simulation](#)
6. [Phase 1 — MoveIt2 & Motion Planning](#)
7. [Phase 2 — MediaPipe Hand Tracking](#)
8. [Phase 3 — YOLOv8 Object Detection](#)
9. [Phase 3 — Camera Calibration & Pixel-to-World](#)
10. [Phase 4 — MQTT & Remote Control](#)
11. [Phase 5 — ESP8266 + PCA9685 Hardware](#)
12. [Phase 5 — ros2_control Hardware Interface](#)
13. [Concept Explainers — Theory You Need](#)
14. [Quick Reference — Commands Cheatsheet](#)

PROF

1. Learning Roadmap — Big Picture

BEFORE EVERYTHING

- |
- └ Linux terminal basics (if shaky)
- └ Python basics (functions, classes, imports)
- └ Git basics (clone, commit, push)

PHASE 0 (done — but review if confused)

- |
- └ ROS2 core concepts (nodes, topics, messages, launch files)
- └ URDF (what it is, how joints/links work)
- └ Gazebo simulation basics

PHASE 1 (current focus)

- |

- | MoveIt2 Setup Assistant (GUI walkthrough)
- | Inverse Kinematics – what IK actually means
- | SRDF + Planning groups concept

PHASE 2

- |
- | MediaPipe hand landmarks (21 points, coordinate system)
- | Mapping math (normalisation, angle computation)

PHASE 3

- |
- | YOLOv8 – how detection works, confidence, bounding boxes
- | Camera calibration – intrinsics, homography, pixel-world

PHASE 4

- |
- | MQTT protocol – pub/sub model
- | Flask web server basics
- | MediaPipe.js in browser

PHASE 5

- |
- | ESP8266 programming (Arduino framework)
- | PCA9685 I2C servo driver
- | ros2_control hardware interface

2. Foundations — What You Must Know Before Anything

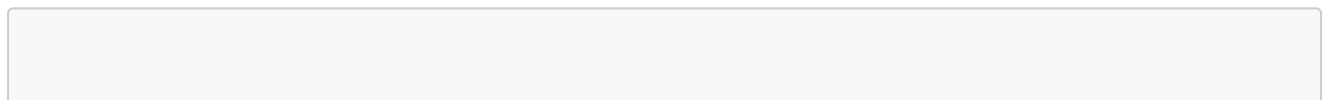
2.1 Linux Terminal (if you need a refresher)

You use this every single day in this project. Must be comfortable with:

PROF

Command	What it does
<code>cd, ls, pwd</code>	Navigate directories
<code>source file.bash</code>	Load environment variables
<code>export VAR=value</code>	Set environment variable
<code>grep, find</code>	Search files and output
<code>Ctrl+C</code>	Kill a running process
<code>ps aux grep ros2</code>	Find running ROS2 processes
<code>pkill -9 -f "gz sim"</code>	Force kill Gazebo

Watch:



YouTube: "Linux Command Line Full Tutorial" by freeCodeCamp
(Just the first hour – file system, navigation, permissions)

2.2 Python You Need for This Project

You don't need advanced Python — but you do need these patterns:

```
# ROS2 nodes are Python classes that inherit from Node
class MyNode(rclpy.node.Node):
    def __init__(self):
        super().__init__('my_node_name')

# Publishers
self.pub = self.create_publisher(JointTrajectory, '/arm_controller', 10)

# Subscribers
self.sub = self.create_subscription(Image, '/camera/image_raw',
self.callback, 10)

# Timers (periodic callbacks)
self.timer = self.create_timer(0.1, self.timer_callback) # 10 Hz
```

Watch:

YouTube: "Python Object Oriented Programming" by Corey Schafer
(Just the Classes video – 45 min)

PROF

3. Phase 0 — ROS2 Fundamentals

Phase 0 is already complete ☒ but if any concept feels unclear, use these resources.

What is ROS2? (30-second version)

ROS2 is a set of tools and libraries that lets multiple Python/C++ programs (called **nodes**) talk to each other by publishing and subscribing to **topics**. Each topic carries messages of a specific **type**.

```
/camera_node  —publishes→  /camera/image_raw  (sensor_msgs/Image)
                                   |
/vision_node  ←subscribes—
```

That's it. Everything in this project is just nodes publishing and subscribing.

Core Concepts to Understand

Concept	One-line explanation	Where used in project
Node	A single running program	<code>camera_node.py</code> , <code>gesture_node.py</code> , etc.
Topic	A named channel for messages	<code>/camera/image_raw</code> , <code>/joint_states</code>
Message	Typed data structure	<code>sensor_msgs/Image</code> , <code>JointTrajectory</code>
Publisher	Node that sends data	<code>camera_node</code> → <code>/camera/image_raw</code>
Subscriber	Node that receives data	<code>vision_node</code> ← <code>/camera/image_raw</code>
Service	Request/response (like a function call)	Movelt2 planning
Action	Long-running task with feedback	Movelt2 execution
Launch file	Script that starts multiple nodes	<code>sim.launch.py</code>
Package	A folder of related nodes + config	<code>robotic_arm_description</code>

YouTube Tutorials — Phase 0 (ROS2)

Priority	Topic	Search
 Essential	ROS2 full beginner course	"ROS2 for Beginners" by Robotics Back-End (full playlist)
 Essential	ROS2 nodes and topics explained	"ROS2 Nodes Topics Services" by Articulated Robotics
 Useful	ROS2 Python publisher/subscriber	"ROS2 Python Publisher Subscriber Tutorial"
 Useful	ROS2 launch files explained	"ROS2 Launch Files Python" by Robotics Back-End
 Reference	<code>ros2_control</code> overview	"ros2_control explained" by Maciej Stępień

PROF

Practice Commands

```
# See all running topics
ros2 topic list

# See what type of message a topic carries
ros2 topic type /joint_states

# Watch messages on a topic in real time
ros2 topic echo /joint_states

# See message structure
```

```
ros2 interface show sensor_msgs/msg/JointState

# See all active nodes
ros2 node list

# See what a node publishes/subscribes to
ros2 node info /robot_state_publisher
```

4. Phase 0 — URDF & Robot Description

What is a URDF?

URDF = **Unified Robot Description Format**. It's an XML file that describes your robot's:

- **Links** — rigid body parts (base, upper arm, forearm, gripper, etc.)
- **Joints** — connections between links (revolute = rotates, fixed = doesn't move)
- **Visuals** — the 3D mesh (STL file) that you see
- **Collision** — simplified geometry for physics simulation
- **Inertia** — mass + moment of inertia (for realistic physics)

```
<!-- Minimal link example -->
<link name="upper_arm">
  <visual>
    <geometry>
      <mesh
filename="package://robotic_arm_description/meshes/upper_arm.stl"/>
      </geometry>
    </visual>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.001" iyy="0.001" izz="0.001" ixy="0" ixz="0"
iyz="0"/>
    </inertial>
  </link>

<!-- Joint connecting base_link to upper_arm -->
<joint name="joint_1" type="revolute">
  <parent link="base_link"/>
  <child link="upper_arm"/>
  <axis xyz="0 0 1"/>          <!-- rotates around Z axis -->
  <limit lower="-1.57" upper="1.57" effort="10" velocity="1.0"/>
</joint>
```

Xacro — URDF with macros

Your robot uses `.urdf.xacro` files. Xacro is just URDF with:

- Variables (`${value}`)

- Macros (reusable blocks)
- File includes (`xacro:include`)

```
# Convert xacro → plain URDF to inspect it
ros2 run xacro xacro robot.urdf.xacro > robot.urdf
check_urdf robot.urdf
```

YouTube Tutorials — URDF

Priority	Topic	Search
 Essential	URDF basics — links and joints	"URDF Tutorial ROS2" by Articulated Robotics
 Essential	URDF xacro tutorial	"Xacro URDF ROS2 tutorial"
 Useful	Understanding inertia tensors	"Robot Inertia URDF explained"
 Useful	Visual vs collision in URDF	"URDF visual collision inertial" by The Construct

Key files in your project

```
src/robotic_arm_description/
├─ urdf/
│   ├── robotic_arm.urdf.xacro    ← main robot description
│   ├── robotic_arm.gazebo        ← Gazebo-specific plugins
│   └─ robotic_arm.trans          ← ros2_control hardware interface
├─ meshes/                       ← 43 STL files
├─ launch/
│   ├── sim.launch.py             ← Gazebo + RViz2
│   └─ display.launch.py          ← RViz2 + joint sliders only
└─ config/
    └─ bridge.yaml                ← ros_gz_bridge topic mapping
```

PROF

5. Phase 0 — Gazebo Simulation

What Gazebo Does

Gazebo is a physics simulator. It:

1. Loads your URDF and treats it as a real physical object
2. Applies gravity, collision, joint torques
3. Publishes sensor data (joint states, camera images) as ROS2 topics

4. Receives control commands (joint velocities, positions) from ROS2

```
Your URDF → Gazebo simulates physics
                |
                |— publishes /joint_states (where joints actually
are)
                |— publishes /camera/image_raw (what camera sees)
                |
                ← subscribes /arm_controller/joint_trajectory (move
command)
```

The Bridge (ros_gz_bridge)

Gazebo Harmonic uses its own internal topic system (not ROS2). The bridge converts between them:

```
# config/bridge.yaml
- ros_topic_name: /joint_states
  gz_topic_name: /world/empty/model/robotic_arm/joint_state
  ros_type_name: sensor_msgs/msg/JointState
  gz_type_name: gz.msgs.Model
  direction: GZ_TO_ROS
```

YouTube Tutorials — Gazebo

Priority	Topic	Search
● Essential	Gazebo Harmonic with ROS2	"Gazebo Harmonic ROS2 Jazzy tutorial"
● Useful	ros_gz_bridge explained	"ros_gz_bridge ROS2 Gazebo"
● Useful	Spawning URDF in Gazebo	"Spawn URDF Gazebo Harmonic ROS2"

PROF

Practice: Inspect your simulation

```
# Source and launch
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robotic_arm_description sim.launch.py

# In another terminal – check topics
ros2 topic list
ros2 topic echo /joint_states --no-arr # watch joint positions
gz topic -l                            # see Gazebo-side topics
```

6. Phase 1 — MoveIt2 & Motion Planning

This is your current active phase. Study this section carefully.

What is MoveIt2?

MoveIt2 is a motion planning framework. Given a **target pose** (where you want the end-effector to be), it:

1. **Solves IK** — computes what joint angles produce that end-effector position
2. **Plans a path** — finds a collision-free sequence of joint positions from current to target
3. **Executes** — sends the trajectory to **ros2_control** to actually move the arm

```
You specify:           MoveIt2 computes:           ros2_control executes:
"Move gripper         → Joint angles that → Smooth trajectory
to XYZ position"      reach that point      sent to arm
```

Inverse Kinematics (IK) — The Core Concept

Forward Kinematics (FK): Given joint angles → where is the end-effector?

Inverse Kinematics (IK): Given end-effector position → what joint angles?

FK is easy (just matrix multiplication). IK is hard (multiple solutions, no guarantee).

MoveIt2 uses solvers like **KDL** or **TRAC-IK** to solve IK for you. You never need to do this math by hand.

```
Example (simplified, 2-joint arm):
Joint 1 = 30°, Joint 2 = 45° → end-effector at (15cm, 8cm) [FK]
end-effector at (15cm, 8cm) → Joint 1 = 30°, Joint 2 = 45° [IK]
```

SRDF — What it is

SRDF (Semantic Robot Description Format) is MoveIt2's companion file to URDF. It stores:

- **Planning groups** — which joints belong to the "arm" group vs "gripper" group
- **Self-collision matrix** — which link pairs can never collide (ignored for speed)
- **Named poses** — "home", "ready", "tucked" positions

The MoveIt2 Setup Assistant generates this file for you.

YouTube Tutorials — MoveIt2

Priority	Topic	Search
Essential	MoveIt2 Setup Assistant walkthrough	"MoveIt2 Setup Assistant ROS2 tutorial" by Robotics Back-End
Essential	MoveIt2 Python interface	"MoveIt2 Python MoveGroupInterface ROS2"

Priority	Topic	Search
● Essential	What is Inverse Kinematics (visual)	"Inverse Kinematics explained visually" by Freya Holmér
● Useful	OMPL path planning intro	"OMPL motion planning explained"
● Useful	MoveIt2 RViz interactive markers	"MoveIt2 RViz2 plan execute"
● Reference	TRAC-IK vs KDL comparison	"TRAC-IK KDL ROS MoveIt comparison"

Direct YouTube Search URLs

```
https://www.youtube.com/results?
search_query=MoveIt2+Setup+Assistant+ROS2+tutorial
https://www.youtube.com/results?
search_query=Inverse+Kinematics+explained+visually
https://www.youtube.com/results?
search_query=MoveIt2+Python+MoveGroupInterface+ROS2
https://www.youtube.com/results?
search_query=OMPL+motion+planning+explained
```

Setup Assistant — Step by Step

1. Launch:


```
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```
2. Load URDF:
 - Click "Load files"
 - Navigate to:


```
src/robotic_arm_description/urdf/robotic_arm.urdf.xacro
```
3. Self-Collision Matrix:
 - Click "Self-Collisions" → "Generate Collision Matrix"
 - Takes 1-2 minutes – finds which link pairs to ignore
4. Planning Groups:
 - Click "Add Group"
 - Name: "arm"
 - Kinematic Solver: "kdl_kinematics_plugin/KDLKinematicsPlugin"
 - Add joints: link_1, link_2, link_3, link_4, link_5
5. End-Effector:
 - Click "Add End Effector"
 - Name: "gripper"
 - End Effector Group: create group with link_6

6. Robot Poses:
 - Add "home" pose – set all joints to 0.0
7. Generate Package:
 - Configuration Files → Browse to: src/robotic_arm_moveit_config
 - Click "Generate Package"
8. Build:
 - colcon build
 - source install/setup.bash
 - ros2 launch robotic_arm_moveit_config demo.launch.py

Practice after Setup Assistant

```
# In RViz2:  
# 1. Set "Planning Group" to "arm"  
# 2. Drag the orange interactive marker sphere (end-effector)  
# 3. Click "Plan" → watch the ghost arm plan the path  
# 4. Click "Execute" → arm should move to target in Gazebo
```

7. Phase 2 — MediaPipe Hand Tracking

What MediaPipe Gives You

MediaPipe's Hand Landmarker returns **21 3D landmarks** per hand, every frame:

Landmark indices:

```
0  = WRIST  
1  = THUMB_CMC  
2  = THUMB_MCP  
3  = THUMB_IP  
4  = THUMB_TIP  
5  = INDEX_FINGER_MCP  
6  = INDEX_FINGER_PIP  
7  = INDEX_FINGER_DIP  
8  = INDEX_FINGER_TIP  
9  = MIDDLE_FINGER_MCP  
...  
20 = PINKY_TIP
```

Each landmark has: x (0→1, left→right), y (0→1, top→bottom), z (depth, relative)

The Mapping Logic You Need to Write

```
# Pseudocode for landmark → joint angle mapping

landmarks = mediapipe_result.hand_landmarks[0] # first hand

# Joint 1 – Base rotation (how far left/right wrist is in frame)
wrist_x = landmarks[0].x # 0 = far left, 1 = far right
joint_1 = map_range(wrist_x, 0.0, 1.0, -1.57, 1.57) # -90° to +90°

# Joint 2 – Shoulder (how far up/down wrist is)
wrist_y = landmarks[0].y
joint_2 = map_range(wrist_y, 0.0, 1.0, 0.0, 1.57)

# Finger curl – measure angle formed by 3 finger landmarks (MCP→PIP→DIP)
def finger_curl_angle(mcp, pip, dip):
    # Two vectors: MCP→PIP and PIP→DIP
    v1 = [pip.x - mcp.x, pip.y - mcp.y]
    v2 = [dip.x - pip.x, dip.y - pip.y]
    # Dot product → cosine → angle
    import math
    cos_a = (v1[0]*v2[0] + v1[1]*v2[1]) / (norm(v1) * norm(v2))
    return math.acos(max(-1, min(1, cos_a))) # radians

# Gripper – thumb-to-index-tip distance
thumb_tip = landmarks[4]
index_tip = landmarks[8]
gap = distance(thumb_tip, index_tip)
joint_6 = map_range(gap, 0.02, 0.3, 0.0, 0.5) # closed to open
```

EMA Filter (Essential — prevents jitter)

```
# Exponential Moving Average – smooths out hand tremor
alpha = 0.3 # lower = smoother, higher = more responsive

def ema_filter(new_value, previous_filtered, alpha=0.3):
    return alpha * new_value + (1 - alpha) * previous_filtered

# Apply to every joint angle before publishing
joint_1_filtered = ema_filter(joint_1_raw, joint_1_filtered)
```

YouTube Tutorials — MediaPipe

Priority	Topic	Search
 Essential	MediaPipe Hand Landmarker Python	"MediaPipe Hand Tracking Python 2024" by Nicholas Renotte
 Essential	MediaPipe landmark indices visual	"MediaPipe 21 hand landmarks explained"

Priority	Topic	Search
● Useful	Angle calculation from landmarks	"hand gesture recognition angle MediaPipe Python"
● Useful	Sending to ROS2 from OpenCV	"OpenCV ROS2 camera node Python"

Direct YouTube Search URLs

```
https://www.youtube.com/results?
search_query=MediaPipe+hand+tracking+Python+2024
https://www.youtube.com/results?
search_query=hand+gesture+recognition+angle+MediaPipe+Python
https://www.youtube.com/results?
search_query=mediapipe+21+landmarks+hand+explained
```

How to Publish to ROS2 from MediaPipe

```
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from builtin_interfaces.msg import Duration

def publish_joints(joint_angles):
    msg = JointTrajectory()
    msg.joint_names = ['link_1', 'link_2', 'link_3', 'link_4', 'link_5',
                      'link_6']

    point = JointTrajectoryPoint()
    point.positions = joint_angles          # list of 6 floats
    (radians)
    point.time_from_start = Duration(nanosec=150_000_000) # 150ms
    minimum

    msg.points = [point]
    publisher.publish(msg)
```

PROF

8. Phase 3 — YOLOv8 Object Detection

What YOLO Does

YOLO (You Only Look Once) processes an entire image in one forward pass through a neural network and returns:

- **Bounding boxes** — `[x_center, y_center, width, height]` in pixels
- **Class label** — what object it is ("bottle", "red_cube", etc.)
- **Confidence score** — 0.0 to 1.0 (how sure the model is)

```
# YOLOv8 usage (already installed)
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # nano model — fastest, ~3MB

# Run on a single frame
results = model(frame)
for box in results[0].boxes:
    x, y, w, h = box.xywh[0] # pixel coordinates
    label = model.names[int(box.cls)] # class name
    conf = float(box.conf) # confidence 0→1
    print(f"{label}: {conf:.2f} at ({x:.0f}, {y:.0f})")
```

Model Sizes (choose based on speed vs accuracy tradeoff)

Model	Size	Speed on RTX 3060	mAP
YOLOv8n (nano)	3MB	~2ms	37.3
YOLOv8s (small)	11MB	~4ms	44.9
YOLOv8m (medium)	25MB	~8ms	50.2
YOLOv8l (large)	43MB	~13ms	52.9

Start with **yolov8n.pt** — it runs at ~500 FPS on your RTX 3060 and is more than fast enough.

YouTube Tutorials — YOLOv8

Priority	Topic	Search
● Essential	YOLOv8 object detection — getting started	"YOLOv8 Object Detection Python Tutorial" by Nicolai Nielsen
● Essential	YOLOv8 custom training	"YOLOv8 Custom Dataset Training Roboflow"
● Useful	YOLOv8 + OpenCV webcam	"YOLOv8 real time webcam detection Python"
● Useful	Getting bounding box coordinates	"YOLOv8 bounding box coordinates Python"
● Reference	CUDA GPU acceleration YOLOv8	"YOLOv8 GPU CUDA Python setup"

Direct YouTube Search URLs

```
https://www.youtube.com/results?
search_query=YOLOv8+Object+Detection+Python+Tutorial
```

```
https://www.youtube.com/results?
search_query=YOLOv8+Custom+Dataset+Training+Roboflow
https://www.youtube.com/results?
search_query=YOLOv8+real+time+webcam+detection+Python
```

Verify CUDA is working

```
# Run this before anything else
import torch
print(torch.cuda.is_available())      # Should print: True
print(torch.cuda.get_device_name(0))  # Should print: NVIDIA GeForce
RTX 3060

from ultralytics import YOLO
model = YOLO('yolov8n.pt')
results = model('https://ultralytics.com/images/bus.jpg')
results[0].show()  # Should open image with bounding boxes
```

9. Phase 3 — Camera Calibration & Pixel-to-World

Why Calibration is Needed

A camera lens introduces distortion — straight lines appear curved. Also, you need to know the camera's **intrinsic parameters** to convert 2D pixel coordinates to 3D world coordinates.

```
Uncalibrated: pixel (450, 320) → ??? in real world
Calibrated:   pixel (450, 320) → (12.3cm, 8.7cm, 0cm) on workspace plane
```

—
PROF

The 3×3 Camera Matrix (what calibration gives you)

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y — focal length in pixels
- c_x, c_y — optical center (usually near image center)

How to Calibrate (OpenCV checkerboard method)

```
# Print a 9x6 checkerboard (search "checkerboard calibration pattern
PDF")
# Hold it in front of your camera at 20+ different angles

# Use ROS2 camera calibration tool
ros2 run camera_calibration cameracalibrator \
  --size 9x6 \
```

```
--square 0.025 \      ← square size in meters (measure your printout)
image:=/camera/image_raw \
camera:=/camera

# Click "Calibrate" when enough samples collected
# Click "Save" → saves camera_info.yaml
```

Pixel → 3D World Coordinate

For the sorting workspace (flat table, known height):

```
import cv2
import numpy as np

# Load calibration
camera_matrix = np.array([[fx, 0, cx],
                          [0, fy, cy],
                          [0, 0, 1]])
dist_coeffs = np.array([k1, k2, p1, p2, k3])

# Given a detected object at pixel (u, v):
# Assumption: all objects on a flat plane Z=0 (workspace height)
# Use homography: calibrate once with known points on workspace

# Method: place 4 known markers on your workspace corners
# Measure their real-world XY coordinates
# Compute homography matrix H
src_pts = np.array([[x1,y1],[x2,y2],[x3,y3],[x4,y4]], dtype=np.float32)
# pixels
dst_pts = np.array([[0,0],[30,0],[30,20],[0,20]], dtype=np.float32)
# real cm

H, _ = cv2.findHomography(src_pts, dst_pts)

# Apply to any detected object pixel
pixel_pt = np.array([[u, v]], dtype=np.float32)
world_pt = cv2.perspectiveTransform(pixel_pt, H)
x_cm, y_cm = world_pt[0][0]
```

PROF

YouTube Tutorials — Camera Calibration

Priority	Topic	Search
 Essential	Camera calibration OpenCV Python	"OpenCV camera calibration Python checkerboard"
 Essential	Homography — pixel to world	"OpenCV homography pixel to world coordinates"

Priority	Topic	Search
● Useful	Perspective transform OpenCV	"findHomography perspectiveTransform OpenCV Python"

10. Phase 4 — MQTT & Remote Control

What MQTT Is

MQTT is a **publish/subscribe messaging protocol** designed for IoT. Instead of direct connections between devices:

Traditional (HTTP):

Phone → directly connects to → Laptop server

Problem: firewall, NAT, needs public IP

MQTT:

Phone → publishes to → HiveMQ Cloud Broker ← → Laptop subscribes

Advantage: both connect OUT to the broker – no ports to open

Core Concepts

Concept	Explanation	Example
Broker	Central message router	HiveMQ Cloud
Topic	Message channel (like a folder path)	<code>arm/joints</code>
Publish	Send a message to a topic	Phone sends <code>{"j1": 0.5, "j2": -0.3 ...}</code>
Subscribe	Listen for messages on a topic	ROS2 bridge receives joint angles
QoS 0	Fire and forget — no guarantee of delivery	Used here: old commands must be dropped

PROF

Python MQTT (paho-mqtt)

```
import paho.mqtt.client as mqtt
import json

# Subscriber (ROS2 bridge side)
def on_message(client, userdata, message):
    data = json.loads(message.payload)
    joint_angles = [data['j1'], data['j2'], data['j3'],
                    data['j4'], data['j5'], data['j6']]
    publish_to_ros2(joint_angles)
```



```

client = mqtt.Client()
client.username_pw_set("your_username", "your_password")
client.on_message = on_message
client.connect("your-broker.hivemq.cloud", 8883)
client.subscribe("arm/joints", qos=0)
client.loop_forever()

```

JavaScript MQTT (phone browser side)

```

// In your HTML page served by Flask + ngrok
const client = mqtt.connect('wss://your-broker.hivemq.cloud:8884/mqtt',
{
  username: 'your_username',
  password: 'your_password'
});

function sendJoints(angles) {
  const payload = JSON.stringify({
    j1: angles[0], j2: angles[1], j3: angles[2],
    j4: angles[3], j5: angles[4], j6: angles[5]
  });
  client.publish('arm/joints', payload, { qos: 0, retain: false });
}

```

YouTube Tutorials — MQTT

Priority	Topic	Search
 Essential	MQTT explained simply	"MQTT explained beginners IoT" by Steve's Internet Guide
 Essential	MQTT Python paho tutorial	"paho-mqtt Python tutorial publish subscribe"
 Useful	HiveMQ Cloud setup	"HiveMQ Cloud free broker setup"
 Useful	Flask web server Python basics	"Flask Python web server beginners" by Tech With Tim
 Useful	ngrok tunnel setup	"ngrok tutorial expose localhost"
 Reference	MQTT.js browser tutorial	"MQTT.js browser JavaScript publish subscribe"

Direct YouTube Search URLs

```

https://www.youtube.com/results?
search_query=MQTT+explained+beginners+IoT

```

```
https://www.youtube.com/results?search_query=paho-mqtt+Python+tutorial
https://www.youtube.com/results?
search_query=Flask+Python+web+server+beginners
https://www.youtube.com/results?
search_query=ngrok+tutorial+expose+localhost
```

11. Phase 5 — ESP8266 + PCA9685 Hardware

ESP8266 — What it is

The ESP8266 NodeMCU is a Wi-Fi microcontroller that you program using Arduino C++. In this project it acts as a **bridge**: receives joint angles from ROS2 (over serial/Wi-Fi), converts them to PWM signals, and sends PWM to the PCA9685.

```
ROS2 Laptop —serial 115200 baud→ ESP8266 —I2C→ PCA9685 —PWM→
Servos
```

PCA9685 — What it is

The PCA9685 is a 16-channel PWM driver controlled over I2C. It generates precise PWM signals for servo control — freeing the ESP8266 from timing-sensitive PWM generation.

Servo PWM spec (MG996R):

- Frequency: 50 Hz (one pulse every 20ms)
- Pulse width: 500µs = fully CW, 1500µs = centre, 2500µs = fully CCW

```
Joint angle (radians) → Pulse width (µs)
```

```
pulse_us = map(angle_rad, -1.57, 1.57, 500, 2500)
```

—
PROF

I2C Addresses

Device	I2C Address	Notes
PCA9685	0x40 (default)	Can be changed via A0-A5 pads
MPU6050	0x68 (default)	ADO pin low
ESP8266	I2C master	No address — it initiates all communication

Arduino Libraries to Install

Arduino IDE → Sketch → Include Library → Manage Libraries:

1. "Adafruit PWM Servo Driver Library" – for PCA9685
2. "Wire" (built-in) – for I2C communication
3. "LowPower" by roocketscream (optional, for sleep mode)

YouTube Tutorials — ESP8266 + PCA9685

Priority	Topic	Search
● Essential	ESP8266 NodeMCU getting started	"ESP8266 NodeMCU Arduino IDE beginner tutorial"
● Essential	PCA9685 servo driver tutorial	"PCA9685 16 channel servo driver Arduino tutorial"
● Essential	I2C explained	"I2C protocol explained simply Arduino"
● Useful	ESP8266 serial communication	"ESP8266 serial communication Arduino"
● Useful	Controlling servos with PCA9685	"control multiple servos PCA9685 I2C Arduino"
● Reference	MPU6050 gyroscope tutorial	"MPU6050 Arduino I2C tutorial gyroscope"

Direct YouTube Search URLs

```
https://www.youtube.com/results?
search_query=ESP8266+NodeMCU+Arduino+IDE+beginner+tutorial
https://www.youtube.com/results?
search_query=PCA9685+16+channel+servo+driver+Arduino
https://www.youtube.com/results?
search_query=I2C+protocol+explained+Arduino
https://www.youtube.com/results?
search_query=MPU6050+Arduino+I2C+tutorial+gyroscope
```

Minimal Firmware Scaffold

```
// ESP8266 firmware – receives joint angles from ROS2, drives PCA9685

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
```

```

#define SERVO_FREQ    50      // 50Hz for standard servos
#define PULSE_MIN     150     // ~500µs (fully clockwise)
#define PULSE_MAX     600     // ~2500µs (fully counter-clockwise)
#define NUM_JOINTS    6

float joint_angles[NUM_JOINTS] = {0, 0, 0, 0, 0, 0};

void setup() {
  Serial.begin(115200);
  Wire.begin(4, 5);          // SDA=GPI04(D2), SCL=GPI05(D1)
  pwm.begin();
  pwm.setPWMFreq(SERVO_FREQ);
  delay(10);
}

void loop() {
  if (Serial.available()) {
    // Expect CSV: "0.5,-0.3,0.8,0.0,-0.5,0.2\n"
    String line = Serial.readStringUntil('\n');
    parseAngles(line);
    for (int i = 0; i < NUM_JOINTS; i++) {
      int pulse = angleToPulse(joint_angles[i]);
      pwm.setPWM(i, 0, pulse);
    }
  }
}

int angleToPulse(float angle_rad) {
  // Map -PI/2..+PI/2 radians to pulse count (150..600)
  float norm = (angle_rad + 1.5708) / 3.1416; // 0→1
  return (int)(PULSE_MIN + norm * (PULSE_MAX - PULSE_MIN));
}

void parseAngles(String csv) {
  int idx = 0;
  while (csv.length() > 0 && idx < NUM_JOINTS) {
    int comma = csv.indexOf(',');
    if (comma == -1) { joint_angles[idx++] = csv.toFloat(); break; }
    joint_angles[idx++] = csv.substring(0, comma).toFloat();
    csv = csv.substring(comma + 1);
  }
}

```

PROF

12. Phase 5 — ros2_control Hardware Interface

What ros2_control Does

ros2_control is the abstraction layer between the **control algorithm** (MoveIt2) and the **actual hardware** (your ESP8266 + servos).

```

MoveIt2 sends:    JointTrajectory (desired positions in radians)
                  |
                  ros2_control
                  SystemInterface
                  read() / write()
                  |
Your hardware interface code
                  |
Serial → ESP8266 → PCA9685 → Servos

```

The key is that **MoveIt2 doesn't know or care** whether `ros2_control` is talking to Gazebo or real hardware. You just swap the hardware interface.

YouTube Tutorials — `ros2_control`

Priority	Topic	Search
● Essential	<code>ros2_control</code> explained from scratch	" <code>ros2_control</code> tutorial beginners 2024" by Articulated Robotics
● Essential	Writing a hardware interface	" <code>ros2_control</code> custom hardware interface <code>SystemInterface</code> "
● Useful	<code>JointTrajectoryController</code> config	" <code>JointTrajectoryController</code> <code>ros2_control</code> YAML"

Direct YouTube Search URLs

```

https://www.youtube.com/results?
search_query=ros2_control+tutorial+beginners+2024
https://www.youtube.com/results?
search_query=ros2_control+custom+hardware+interface+SystemInterface

```

PROF

13. Concept Explainers — Theory You Need

What is a Transformation Frame (TF)?

Every link in the robot has a coordinate frame. TF tracks where each frame is relative to the world. When RViz2 shows "No transform from `base_link` to `upper_arm`", it means it doesn't know where these frames are relative to each other.

```

# Visualise the TF tree
ros2 run tf2_tools view_frames      # generates PDF
ros2 run rviz2 rviz2               # TF display in RViz2

```

Watch:

YouTube: "ROS2 TF2 Transforms explained" by Articulated Robotics

What is a Quaternion?

Orientation in 3D cannot be represented as just X/Y/Z angles (Euler angles have gimbal lock). ROS2 uses **quaternions** (4 numbers: x, y, z, w) to represent orientation.

You will see them in `geometry_msgs/Pose` and `sensor_msgs/Imu`.

```
# Convert Euler angles (roll, pitch, yaw) to quaternion
from scipy.spatial.transform import Rotation
r = Rotation.from_euler('xyz', [roll, pitch, yaw])
q = r.as_quat()    # [x, y, z, w]
```

Watch:

YouTube: "Quaternions and 3d rotation, explained interactively" by 3Blue1Brown

What is PID Control?

In the future, if you add closed-loop servo control, you'll use PID.

- **P** (Proportional): Correction proportional to current error
- **I** (Integral): Correction based on accumulated past error
- **D** (Derivative): Correction based on rate of change of error

For now, everything uses **open-loop** (commanded position sent, no verification). This is why MG996R servos can droop — they're open-loop.

14. Quick Reference — Commands Cheatsheet

```
# — Environment —————
source /opt/ros/jazzy/setup.bash
source install/setup.bash
colcon build --symlink-install
colcon build --packages-select robotic_arm_description

# — Kill everything cleanly —————
```

```
pkill -9 -f "gz sim"; pkill -9 -f ruby; pkill -9 -f ros2; pkill -9 -f rviz
```

```
# — Launch —————  
ros2 launch robotic_arm_description sim.launch.py  
ros2 launch robotic_arm_description display.launch.py  
ros2 launch robotic_arm_moveit_config demo.launch.py
```

```
# — Inspect —————  
ros2 topic list  
ros2 topic echo /joint_states --no-arr  
ros2 topic hz /camera/image_raw  
ros2 node list  
ros2 node info /move_group  
ros2 interface show trajectory_msgs/msg/JointTrajectory
```

```
# — URDF —————  
ros2 run xacro xacro robot.urdf.xacro > /tmp/robot.urdf  
check_urdf /tmp/robot.urdf
```

```
# — Gazebo —————  
gz topic -l # list Gazebo topics  
gz topic -e -t /world/empty/model/robotic_arm/joint_state
```

```
# — Python quick test —————  
python3 -c "import torch; print(torch.cuda.is_available())"  
python3 -c "from ultralytics import YOLO; print('YOLOv8 OK')"  
python3 -c "import mediapipe; print('MediaPipe OK')"  
python3 -c "import cv2; print(cv2.__version__)"
```

Learning Order Summary

Week 1: ROS2 fundamentals → URDF → Gazebo
└ Already done  – review videos if confused

Week 2: MoveIt2 Setup Assistant → IK concept → Plan+Execute in RViz2
└ CURRENT FOCUS ← start here now

Week 3: MediaPipe mapping math → gesture_node.py → live sim control

Week 4: YOLOv8 → camera calibration → sorting pipeline

Week 5: MQTT + Flask + ngrok → remote control

Week 6: ESP8266 firmware → PCA9685 wiring → ros2_control hardware interface

*Use this guide alongside the PROGRESS.md checklist.
Learn each section just before you start that phase — not all at once.*